

Python Collections - ausführlicher Spickzettel

Listen · Dictionaries · Sets · Tupel

Benjamin Malicke ©

Januar 2026

Inhaltsverzeichnis

1	Listen	3
1.1	Wichtigste Listenmethoden - Beispiele	3
1.2	Ergaenzend: Slicing & sorted	4
1.3	Kompaktes Listen-Spickzettel-Skript	4
2	Kompakte Tabellen - Listen	6
3	Dictionaries	7
4	Sets	8
5	Tupel	8
6	Vergleich der vier Collection-Typen	8
7	Dictionaries	9
7.1	vollstaendiges Skript: dict_spickzettel.py	9
7.2	Kurze uebersicht - wichtigste Dictionary-Methoden	11
7.3	Wichtiger Hinweis zu copy()	12
8	Sets	12
8.1	Vollstaendiges Skript: set_spickzettel.py	12
8.2	Kurze uebersicht - wichtigste Set-Methoden & Operatoren	16
9	Tupel	16
9.1	Vollstaendiges Skript: tuple_spickzettel.py	16
9.2	Kurze uebersicht - Tupel-Eigenschaften & typische Anwendungsfaelle	19
10	Wann welchen Collection-Typ waehlen? - Entscheidungshilfe & Beispiele	22
10.1	Entscheidungsbaum - Schnelluebersicht	22
10.2	Detaillierte Anwendungsfaelle mit Code-Beispielen	22
10.2.1	1. list - wenn Reihenfolge + aenderbarkeit wichtig sind	22
10.2.2	2. dict - wenn Zugriff per Schluessel wichtig ist	23
10.2.3	3. set - wenn Einzigartigkeit oder Mengenoperationen zaehlen	24
10.2.4	4. tuple - wenn Daten fest / unveraenderlich sein sollen	24
10.3	Zusammenfassung - typische Kombinationen in der Praxis	25
11	Verschachtelte Collections (Nested Structures) - Wichtig & tueckisch	25
11.1	1. Das klassische Problem: Shallow Copy reicht nicht	25
11.2	2. Loesung: Deep Copy	26

11.3	3. Typische verschachtelte Strukturen - Beispiele	26
11.3.1	Liste von Dictionaries (sehr haeufig: JSON-aehnlich)	26
11.3.2	Dictionary mit Listen als Werten (Gruppierung)	26
11.3.3	Matrix (Liste von Listen)	26
11.3.4	Tupel als Schluessel in Dicts (sehr nuetzlich)	27
11.4	4. Schnelle Merkgeregeln fuer verschachtelte Collections	27
11.5	5. Zusammenfassung - typische Fehlerquellen	27

1 Listen

1.1 Wichtigste Listenmethoden - Beispiele

Hier zu jeder wichtigen Listenmethode ein kurzes, lauffaehiges Beispiel mit Kommentar zum Rueckgabewert.

```
1 # 1. append(x)
2 lst = [1, 2, 3]
3 ret = lst.append(4)
4 print(lst)      # [1, 2, 3, 4]
5 print(ret)      # None
```

```
1 # 2. extend(iterable)
2 lst = [1, 2]
3 ret = lst.extend([3, 4])
4 print(lst)      # [1, 2, 3, 4]
5 print(ret)      # None
```

```
1 # 3. insert(i, x)
2 lst = [1, 3, 4]
3 ret = lst.insert(1, 2)
4 print(lst)      # [1, 2, 3, 4]
5 print(ret)      # None
```

```
1 # 4. remove(x)
2 lst = [1, 2, 3, 2]
3 ret = lst.remove(2)
4 print(lst)      # [1, 3, 2]
5 print(ret)      # None
6 # lst.remove(99) # ValueError
```

```
1 # 5. clear()
2 lst = [1, 2, 3]
3 ret = lst.clear()
4 print(lst)      # []
5 print(ret)      # None
```

```
1 # 6. sort(key=None, reverse=False)
2 lst = [3, 1, 2]
3 ret = lst.sort()
4 print(lst)      # [1, 2, 3]
5 print(ret)      # None
6
7 # mit key und reverse
8 lst = ["aaa", "b", "cc"]
9 lst.sort(key=len, reverse=True)
10 print(lst)      # ['aaa', 'cc', 'b']
```

```
1 # 7. reverse()
2 lst = [1, 2, 3]
3 ret = lst.reverse()
4 print(lst)      # [3, 2, 1]
5 print(ret)      # None
```

```
1 # 8. pop(i=-1)
2 lst = [10, 20, 30]
3 ret1 = lst.pop()      # letztes Element
```

```

4 print(ret1)           # 30
5 print(lst)           # [10, 20]
6
7 ret2 = lst.pop(0)     # erstes Element
8 print(ret2)          # 10
9 print(lst)           # [20]

```

```

1 # 9. copy() - flache Kopie
2 lst = [1, 2, 3]
3 cp = lst.copy()
4 print(cp) # [1, 2, 3]
5 print(cp is lst) # False

```

```

1 # 10. count(x)
2 lst = [1, 2, 2, 3]
3 ret = lst.count(2)
4 print(ret)           # 2
5 print(lst)           # [1, 2, 2, 3]

```

```

1 # 11. index(x[, start[, end]])
2 lst = [10, 20, 30, 20]
3 ret1 = lst.index(20)
4 print(ret1)          # 1
5
6 ret2 = lst.index(20, 2)
7 print(ret2)          # 3
8 # lst.index(99)      # ValueError

```

1.2 Ergaenzend: Slicing & sorted

```

1 lst = [0, 1, 2, 3, 4]
2 part = lst[1:4]
3 print(part)          # [1, 2, 3]
4
5 cp = lst[:]
6 print(cp)            # [0, 1, 2, 3, 4]
7 print(cp is lst)     # False

```

```

1 lst = [3, 1, 2]
2 new = sorted(lst)
3 print(new)           # [1, 2, 3]
4 print(lst)           # [3, 1, 2] (unveraendert)

```

1.3 Kompaktes Listen-Spickzettel-Skript

```

1 # listen_spickzettel.py
2 print("=====")
3 print(" LISTEN-SPICKZETTEL ")
4 print("=====\n")
5
6 # 1. append(x)
7 print("1) append(x)")
8 lst = [1, 2, 3]
9 ret = lst.append(4)
10 print("lst:", lst)
11 print("ret:", ret)
12 print("-")

```

```
13
14 # 2. extend(iterable)
15 print("2) extend(iterable)")
16 lst = [1, 2]
17 ret = lst.extend([3, 4])
18 print("lst:", lst) # [1, 2, 3, 4]
19 print("ret:", ret) # None
20 print("-")
21
22 # 3. insert(i, x)
23 print("3) insert(i, x)")
24 lst = [1, 3, 4]
25 ret = lst.insert(1, 2)
26 print("lst:", lst) # [1, 2, 3, 4]
27 print("ret:", ret) # None
28 print("-")
29
30 # 4. remove(x)
31 print("4) remove(x)")
32 lst = [1, 2, 3, 2]
33 ret = lst.remove(2)
34 print("lst:", lst) # [1, 3, 2]
35 print("ret:", ret) # None
36 # Achtung: lst.remove(99) -> ValueError, wenn 99 nicht vorhanden ist
37 print("-")
38
39 # 5. clear()
40 print("5) clear()")
41 lst = [1, 2, 3]
42 ret = lst.clear()
43 print("lst:", lst) # []
44 print("ret:", ret) # None
45 print("-")
46
47 # 6. sort(key=None, reverse=False)
48 print("6) sort(key=None, reverse=False)")
49 lst = [3, 1, 2]
50 ret = lst.sort()
51 print("lst:", lst) # [1, 2, 3]
52 print("ret:", ret) # None
53
54 lst = ["aaa", "b", "cc"]
55 lst.sort(key=len, reverse=True)
56 print("lst mit key=len, reverse=True:", lst) # ['aaa', 'cc', 'b']
57 print("-")
58
59 # 7. reverse()
60 print("7) reverse()")
61 lst = [1, 2, 3]
62 ret = lst.reverse()
63 print("lst:", lst) # [3, 2, 1]
64 print("ret:", ret) # None
65 print("-")
66
67 # 8. pop(i=-1)
68 print("8) pop(i=-1)")
69 lst = [10, 20, 30]
70 ret1 = lst.pop() # letztes Element
71 print("ret1:", ret1) # 30
72 print("lst:", lst) # [10, 20]
```

```
73
74 ret2 = lst.pop(0) # erstes Element
75 print("ret2:", ret2) # 10
76 print("lst:", lst) # [20]
77 print("-")
78
79 # 9. copy() - flache Kopie
80 print("9) copy() - flache Kopie")
81 lst = [1, 2, 3]
82 cp = lst.copy()
83 print("lst:", lst)
84 print("cp:", cp)
85 print("cp is lst:", cp is lst) # False (anderes Objekt)
86 print("-")
87
88 # 10. count(x)
89 print("10) count(x)")
90 lst = [1, 2, 2, 3]
91 ret = lst.count(2)
92 print("lst:", lst)
93 print("count(2):", ret) # 2
94 print("-")
95
96 # 11. index(x[, start[, end]])
97 print("11) index(x[, start[, end]])")
98 lst = [10, 20, 30, 20]
99 ret1 = lst.index(20)
100 print("index(20):", ret1) # 1
101
102 ret2 = lst.index(20, 2) # Suche ab Index 2
103 print("index(20, 2):", ret2) # 3
104 # Achtung: lst.index(99) -> ValueError, wenn 99 nicht vorhanden ist
105 print("-")
106
107 # 12. Slicing - neue Liste
108 print("12) Slicing - neue Liste")
109 lst = [0, 1, 2, 3, 4]
110 part = lst[1:4]
111 print("lst:", lst)
112 print("lst[1:4]:", part) # [1, 2, 3]
113
114 cp = lst[:]
115 print("lst[:]:", cp)
116 print("cp is lst:", cp is lst) # False
117 print("-")
118
119 # 13. sorted(iterable) - neue sortierte Liste
120 print("13) sorted(iterable)")
121 lst = [3, 1, 2]
122 new = sorted(lst)
123 print("lst:", lst) # [3, 1, 2]
124 print("new:", new) # [1, 2, 3]
125 print("-")
```

Listing 1: listen_spickzettel.py

2 Kompakte Tabellen - Listen

Methoden	ändert Liste?	Rückgabewert	Kurzbeschreibung
<code>append(x)</code>	ja	None	hängt ein Element hinten an
<code>extend(iterable)</code>	ja	None	hängt alle Elemente eines Iterables an
<code>insert(i, x)</code>	ja	None	fügt an Position i ein
<code>remove(x)</code>	ja	None / ValueError	löscht erstes Vorkommen von x
<code>clear()</code>	ja	None	leert die Liste
<code>sort(...)</code>	ja	None	sortiert die Liste
<code>reverse()</code>	ja	None	kehrt die Reihenfolge um
<code>pop(i=-1)</code>	ja	entferntes Element	entfernt und liefert Element
<code>copy()</code>	nein	neue Liste	flache Kopie
<code>count(x)</code>	nein	int	Anzahl der Vorkommen
<code>index(...)</code>	nein	int / ValueError	Index des ersten Vorkommens

Ergänzende Operationen:

Operation	ändert Liste?	Rückgabewert	Kurzbeschreibung
<code>lst[:]</code>	nein	neue Liste	flache Kopie per Slice
<code>lst[a:b]</code>	nein	neue Liste	Teilstück
<code>sorted(iterable)</code>	nein	neue sortierte Liste	Sortierte Kopie

3 Dictionaries

Hier nur ein Auszug - das komplette `dict_spickzettel.py`-Skript ist sehr lang. Du kannst es 1:1 in eine `lstlisting`-Umgebung kopieren.

Wichtige Methoden kurz:

- `get(key, default=None)` - sicherer Zugriff
- `keys()` / `values()` / `items()` - Ansichten
- `update(other)` - mehrere Werte übernehmen (None)
- `pop(key[, default])` - entfernt + liefert Wert
- `popitem()` - entfernt letztes Paar (ab 3.7)
- `setdefault(key[, default])` - holen oder setzen
- `clear()` - alles löschen
- `copy()` - flache Kopie
- `fromkeys(...)` - neues Dict aus Schlüsseln

Das vollständige Skript (13 Punkte + Verschachtelung + Iteration) kannst du direkt übernehmen.

4 Sets

Auszug aus set_spickzettel.py:

```

1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 print(A | B)           # Vereinigung
5 print(A & B)           # Schnitt
6 print(A - B)           # Differenz
7 print(A ^ B)           # Symmetrische Differenz
8
9 C = A.copy()
10 C |= B                 # in-place Vereinigung

```

Wichtige Unterschiede: `remove()` -> `KeyError`, `discard()` -> kein Fehler

5 Tupel

Wichtigste Merkmale:

- immutable -> keine Änderung nach Erstellung
- `(42,)` <- Komma zwingend bei Ein-Element-Tupel
- sehr häufig für: Mehrfach-Rückgabewerte, Unpacking, Dict-Keys

Beispiel Unpacking:

```

1 values = (1, 2, 3, 4, 5)
2 first, *middle, last = values
3 print(middle)           # [2, 3, 4] <-- Liste!

```

6 Vergleich der vier Collection-Typen

Typ	Geordnet?	änderbar?	Duplikate?	Zugriff	Typische Nutzung
list	ja	ja	ja	Index	veränderliche Reihenfolgen
dict	ja*	ja	Keys nein	Schlüssel	key -> value Zuordnungen
set	nein	ja	nein	in-Test	Einzigartigkeit, Mengenoperationen
tuple	ja	nein	ja	Index	feste Strukturen, Rückgabewerte

* ab Python 3.7 garantiert

Wann was?

Wann nehme ich was?

Liste - wenn Reihenfolge + Änderungen wichtig sind

Dict - wenn Zugriff per Namen/Schlüssel nötig

Set - wenn Einzigartigkeit + schnelle „ist drin?“-Tests

Tuple - wenn Daten unveränderlich sein sollen / als Schlüssel / Rückgabewerte

7 Dictionaries

Hier ein kompaktes Spickzettel-Skript fuer Dictionaries in Python - mit den wichtigsten Methoden, Rueckgabewerten und typischen Fallen.

7.1 vollstaendiges Skript: dict_spickzettel.py

```

1  # dict_spickzettel.py
2
3  print("=====")
4  print(" DICT-SPICKZETTEL")
5  print("=====\\n")
6
7  # Basis: Dict anlegen
8  print("0) Dict-Grundlagen")
9  user = {"name": "Alice", "age": 30}
10 print("user:", user)
11 print("Zugriff: user['name'] ->", user["name"])
12 print("Schluessel-Liste: list(user.keys()) ->", list(user.keys()))
13 print("-")
14
15 # 1. get(key, default=None)
16 print("1) get(key, default=None)")
17 user = {"name": "Alice", "age": 30}
18 print("user.get('name'):", user.get("name")) # 'Alice'
19 print("user.get('city'):", user.get("city")) # None
20 print("user.get('city', 'unbekannt'):", user.get("city", "unbekannt"))
21 print("-")
22
23 # 2. keys() / values() / items()
24 print("2) keys(), values(), items()")
25 user = {"name": "Alice", "age": 30}
26 print("keys():", list(user.keys())) # ['name', 'age']
27 print("values():", list(user.values())) # ['Alice', 30]
28 print("items():", list(user.items())) # [('name', 'Alice'), ('age', 30)]
29 print("Typ von keys():", type(user.keys()))
30 print("-")
31
32 # 3. in-Operator (Schluessel-Pruefung)
33 print("3) 'in' fuer Schluessel")
34 user = {"name": "Alice", "age": 30}
35 print("'name' in user:", "name" in user) # True
36 print("'city' in user:", "city" in user) # False
37 print("-")
38
39 # 4. Zuweisung / ueberschreiben
40 print("4) Zuweisung / ueberschreiben")
41 user = {"name": "Alice", "age": 30}
42 user["city"] = "Berlin" # neuer Schluessel
43 user["age"] = 31 # ueberschreibt bestehenden
44 print("user:", user)
45 print("-")
46
47 # 5. update(other_dict) - in-place, Rueckgabewert None
48 print("5) update(other_dict)")
49 user = {"name": "Alice", "age": 30}
50 extra = {"city": "Berlin", "age": 31}
51 ret = user.update(extra)
52 print("user nach update:", user) # age ueberschrieben, city neu
53 print("ret:", ret) # None

```

```
54 print("-")
55
56 # 6. pop(key[, default]) - entfernt Schluessel und gibt Wert zurueck
57 print("6) pop(key[, default])")
58 user = {"name": "Alice", "age": 30}
59 val = user.pop("age")
60 print("entfernter Wert:", val) # 30
61 print("user:", user) # {'name': 'Alice'}
62
63 # mit default, wenn Schluessel fehlt
64 val2 = user.pop("city", "nicht vorhanden")
65 print("pop('city', 'nicht vorhanden'):", val2)
66 print("user:", user)
67 # user.pop('city') # ohne default -> KeyError, wenn fehlt
68 print("-")
69
70 # 7. popitem() - entfernt und gibt *ein* (key, value)-Paar zurueck
71 print("7) popitem()")
72 user = {"name": "Alice", "age": 30}
73 item = user.popitem()
74 print("entferntes Paar:", item) # z.B. ('age', 30) (in neueren Pythons:
    letztes Paar)
75 print("user:", user)
76 print("-")
77
78 # 8. setdefault(key[, default]) - Wert holen oder setzen
79 print("8) setdefault(key[, default])")
80 user = {"name": "Alice"}
81 val1 = user.setdefault("name", "Standard")
82 print("val1 (name schon da):", val1) # 'Alice'
83 print("user:", user)
84
85 val2 = user.setdefault("city", "Berlin")
86 print("val2 (city neu gesetzt):", val2) # 'Berlin'
87 print("user:", user) # {'name': 'Alice', 'city': 'Berlin'}
88 print("-")
89
90 # 9. clear() - alles loeschen, Rueckgabewert None
91 print("9) clear()")
92 user = {"name": "Alice", "age": 30}
93 ret = user.clear()
94 print("user:", user) # {}
95 print("ret:", ret) # None
96 print("-")
97
98 # 10. copy() - flache Kopie
99 print("10) copy() - flache Kopie")
100 user = {"name": "Alice", "age": 30}
101 cp = user.copy()
102 print("user:", user)
103 print("cp:", cp)
104 print("cp is user:", cp is user) # False (anderes Dict)
105 print("-")
106
107 # 11. fromkeys(iterable, value=None) - neues Dict erzeugen
108 print("11) fromkeys(iterable, value=None)")
109 keys = ["a", "b", "c"]
110 new_dict = dict.fromkeys(keys, 0)
111 print("new_dict:", new_dict) # {'a': 0, 'b': 0, 'c': 0}
112 print("-")
```

```

113
114 # 12. Verschachtelte Dicts und flache copy()
115 print("12) Verschachtelte Dicts und copy()")
116 config = {
117     "db": {"host": "localhost", "port": 5432},
118     "debug": True,
119 }
120 config_cp = config.copy() # nur aeussere Ebene kopiert
121
122 print("config['db'] is config_cp['db']:", config["db"] is
123       config_cp["db"]) # True
124
125 config["db"]["port"] = 5433
126 print("config:", config)
127 print("config_cp:", config_cp) # innere Dicts werden mit geaendert
128 print("-")
129
130 # 13. Iteration ueber Dicts
131 print("13) Iteration ueber Dicts")
132 user = {"name": "Alice", "age": 30, "city": "Berlin"}
133
134 print("ueber keys():")
135 for k in user.keys():
136     print(" key:", k)
137
138 print("ueber values():")
139 for v in user.values():
140     print(" value:", v)
141
142 print("ueber items():")
143 for k, v in user.items():
144     print(f" {k} -> {v}")
145
146 print("-")
147 print("Fertig mit Dict-Spickzettel.")

```

Listing 2: dict_spickzettel.py - Komplettes Spickzettel-Skript

7.2 Kurze uebersicht - wichtigste Dictionary-Methoden

Methode / Operation	aendert Dict?	Rueckgabewert	Kurzbeschreibung / typische Falle
get(key, default=None)	nein	Wert oder default	Sicherer Zugriff - vermeidet KeyError
keys() / values() / items()	nein	dict_view	Ansichten (sehr effizient)
in	nein	bool	Prueft, ob Schluessel existiert
d[key] = value	ja	-	Neuen Schluessel setzen oder ueberschreiben
update(other)	ja	None	Mehrere Schluessel/Werte uebernehmen
pop(key[, default])	ja	Wert oder default	Entfernt Schluessel - KeyError ohne default
popitem()	ja	(key, value)	Entfernt und gibt ein Paar zurueck (meist letztes)

setdefault(key[, default])	ja	Wert	Holt Wert oder setzt Default und gibt ihn zurueck
clear()	ja	None	Entfernt alle Eintraege
copy()	nein	neues Dict	Flache Kopie - Achtung bei verschachtelten Objekten!
fromkeys(iterable, v=None)	nein	neues Dict	Erzeugt Dict mit gleichen Werten fuer alle Schluessel

7.3 Wichtiger Hinweis zu copy()

Bei verschachtelten Dictionaries kopiert `copy()` nur die aeussere Ebene (shallow copy). aenderungen an inneren Objekten wirken sich auf beide Dictionaries aus.

```

1 config = {"db": {"port": 5432}}
2 config_cp = config.copy()
3 config["db"]["port"] = 5433
4 print(config_cp["db"]["port"])    # auch 5433 !

```

Fuer eine tiefe Kopie nutze `copy.deepcopy()` aus dem `copy`-Modul.

8 Sets

Hier ein kompakter Spickzettel zu Sets in Python - als ausfuehrbares Skript, das Sie z.B. als `set_spickzettel.py` speichern und ausfuehren koennen.

8.1 Vollstaendiges Skript: set_spickzettel.py

```

1 # set_spickzettel.py
2
3 print("=====")
4 print(" SET-SPICKZETTEL")
5 print("=====\n")
6
7 # 0) Grundlagen: Set anlegen
8 print("0) Grundlagen: Set anlegen")
9
10 s = {1, 2, 3}
11 print("s:", s)
12 print("Typ:", type(s))
13
14 # Leeres Set: set(), NICHT {} ({} ist ein leeres dict)
15 empty = set()
16 print("leeres Set:", empty, "Typ:", type(empty))
17 print("-")
18
19 # 1) Einzigartigkeit & Unordnung
20 print("1) Einzigartigkeit & Unordnung")
21
22 s = {1, 2, 2, 3, 3, 3}
23 print("{1, 2, 2, 3, 3, 3} ->", s) # doppelte Werte verschwinden
24
25 s2 = {3, 1, 2}
26 print("{3, 1, 2} ->", s2, "(Reihenfolge beliebig)")
27 print("-")
28
29 # 2) Mitgliedschafts-Test mit 'in'
30 print("2) 'in' fuer Mitgliedschaft")

```

```
31
32 s = {1, 2, 3}
33 print("2 in s:", 2 in s) # True
34 print("5 in s:", 5 in s) # False
35 print("-")
36
37 # 3) add(x) - Element hinzufuegen
38 print("3) add(x)")
39
40 s = {1, 2}
41 ret = s.add(3)
42 print("s nach add(3):", s) # {1, 2, 3}
43 print("Rueckgabewert:", ret) # None
44
45 # doppelt hinzufuegen macht nichts kaputt
46 s.add(3)
47 print("s nach erneutem add(3):", s)
48 print("-")
49
50 # 4) update(iterable) - mehrere Elemente hinzufuegen
51 print("4) update(iterable)")
52
53 s = {1, 2}
54 ret = s.update([2, 3, 4])
55 print("s nach update([2,3,4]):", s) # {1, 2, 3, 4}
56 print("Rueckgabewert:", ret) # None
57 print("-")
58
59 # 5) remove(x) vs discard(x)
60 print("5) remove(x) vs discard(x)")
61
62 s = {1, 2, 3}
63 ret = s.remove(2)
64 print("s nach remove(2):", s) # {1, 3}
65 print("Rueckgabewert:", ret) # None
66
67 # s.remove(5) # -> KeyError, wenn 5 nicht drin ist
68
69 s = {1, 2, 3}
70 ret = s.discard(2)
71 print("s nach discard(2):", s) # {1, 3}
72 print("Rueckgabewert:", ret) # None
73
74 # discard wirft KEINEN Fehler, wenn Element fehlt:
75 ret = s.discard(5)
76 print("s nach discard(5) (nicht vorhanden):", s)
77 print("Rueckgabewert:", ret) # None
78 print("-")
79
80 # 6) pop() - entfernt ein "beliebiges" Element
81 print("6) pop() - entfernt ein beliebiges Element")
82
83 s = {1, 2, 3}
84 val = s.pop()
85 print("entferntes Element:", val)
86 print("Set danach:", s)
87
88 # Achtung: Reihenfolge ist NICHT definiert
89 print("-")
90
```

```
91 # 7) clear() - Set leeren
92 print("7) clear()")
93
94 s = {1, 2, 3}
95 ret = s.clear()
96 print("s nach clear():", s) # set()
97 print("Rueckgabewert:", ret) # None
98 print("-")
99
100 # 8) copy() - flache Kopie
101 print("8) copy() - flache Kopie")
102
103 s = {1, 2, 3}
104 cp = s.copy()
105 print("s:", s)
106 print("cp:", cp)
107 print("cp is s:", cp is s) # False
108 print("-")
109
110 # 9) Mengenoperationen: Vereinigung, Schnitt, Differenz, symm. Differenz
111 print("9) Mengenoperationen (union, intersection, difference,
112       symmetric_difference)")
113
114 A = {1, 2, 3}
115 B = {3, 4, 5}
116 print("A:", A)
117 print("B:", B)
118
119 # 9.1) Vereinigung - alle Elemente aus A und B
120 print("9.1) union / |")
121
122 u1 = A.union(B)
123 u2 = A | B
124 print("A.union(B):", u1) # {1, 2, 3, 4, 5}
125 print("A | B:", u2)
126
127 # 9.2) Durchschnitt (Schnittmenge)
128 print("9.2) intersection / &")
129
130 i1 = A.intersection(B)
131 i2 = A & B
132 print("A.intersection(B):", i1) # {3}
133 print("A & B:", i2)
134
135 # 9.3) Differenz (in A, aber nicht in B)
136 print("9.3) difference / -")
137
138 d1 = A.difference(B)
139 d2 = A - B
140 print("A.difference(B):", d1) # {1, 2}
141 print("A - B:", d2)
142
143 # 9.4) Symmetrische Differenz (in genau einem von beiden)
144 print("9.4) symmetric_difference / ^")
145
146 sd1 = A.symmetric_difference(B)
147 sd2 = A ^ B
148 print("A.symmetric_difference(B):", sd1) # {1, 2, 4, 5}
149 print("A ^ B:", sd2)
150 print("-")
```

```

150
151 # 10) In-Place-Varianten der Mengenoperationen
152 print("10) In-Place-Varianten: |=, &=, -=, ^=")
153
154 A = {1, 2, 3}
155 B = {3, 4, 5}
156
157 C = A.copy()
158 C |= B # C = C.union(B)
159 print("C nach |= B:", C)
160
161 C = A.copy()
162 C &= B # C = C.intersection(B)
163 print("C nach &= B:", C)
164
165 C = A.copy()
166 C -= B # C = C.difference(B)
167 print("C nach -= B:", C)
168
169 C = A.copy()
170 C ^= B # C = C.symmetric_difference(B)
171 print("C nach ^= B:", C)
172 print("-")
173
174 # 11) issubset, issuperset, isdisjoint
175 print("11) issubset, issuperset, isdisjoint")
176
177 A = {1, 2}
178 B = {1, 2, 3}
179 C = {4, 5}
180
181 print("A:", A)
182 print("B:", B)
183 print("C:", C)
184
185 print("A.issubset(B):", A.issubset(B)) # True
186 print("B.issuperset(A):", B.issuperset(A)) # True
187 print("A.isdisjoint(C):", A.isdisjoint(C)) # True (keine gemeinsamen
    Elemente)
188 print("-")
189
190 # 12) frozenset - unveraenderliche Menge
191 print("12) frozenset - unveraenderliche Menge")
192
193 fs = frozenset([1, 2, 3])
194 print("fs:", fs)
195 print("Typ:", type(fs))
196
197 # fs.add(4) # wuerde AttributeError werfen (nicht erlaubt)
198
199 # Aber: Mengenoperationen funktionieren trotzdem und liefern neue Sets
200 fs2 = frozenset([3, 4])
201 print("fs | fs2:", fs | fs2) # neues (normales) set/frozenset
202 print("-")
203
204 print("Fertig mit Set-Spickzettel.")

```

Listing 3: set_spickzettel.py - Komplettes Spickzettel-Skript fuer Sets

8.2 Kurze uebersicht - wichtigste Set-Methoden & Operatoren

Methode / Operator	aendert Set?	Rueckgabewert	Bemerkung
add(x)	ja	None	Fuegt Element hinzu (falls nicht vorhanden)
update(iterable) / =	ja	None	Fuegt mehrere Elemente hinzu
discard(x)	ja	None	Entfernt Element (kein Fehler, wenn nicht vorhanden)
remove(x)	ja	None	Entfernt Element (KeyError , wenn nicht vorhanden)
pop()	ja	beliebiges Element	Entfernt und gibt ein Element zurueck
clear()	ja	None	Leert das Set komplett
copy()	nein	neues Set	Flache Kopie
union /	nein	neues Set	$A \cup B$
intersection / &	nein	neues Set	$A \cap B$
difference / -	nein	neues Set	$A \setminus B$
symmetric_difference / ^	nein	neues Set	$(A \setminus B) \cup (B \setminus A)$
issubset / <=	nein	bool	$A \subseteq B?$
issuperset / >=	nein	bool	$A \supseteq B?$
isdisjoint()	nein	bool	$A \cap B = \emptyset?$
frozenset(...)	-	-	Immutable Version eines Sets

9 Tupel

Hier ein Spickzettel zu Tuples in Python als ausfuehrbares Skript - z. B. als `tuple_spickzettel.py` speichern.

9.1 Vollstaendiges Skript: tuple_spickzettel.py

```

1  # tuple_spickzettel.py
2
3  print("=====")
4  print(" TUPLE-SPICKZETTEL")
5  print("=====\n")
6
7  # 0) Grundlagen: Tuple anlegen
8  print("0) Grundlagen: Tuple anlegen")
9
10 # normales Tuple
11 t = (1, 2, 3)
12 print("t:", t, "Typ:", type(t))
13
14 # Klammern sind optional (Achtung Lesbarkeit)
15 t2 = 4, 5, 6
16 print("t2:", t2, "Typ:", type(t2))
17
18 # EIN-Element-Tuple: Komma ist Pflicht!
19 one_val = (42,)
20 not_tuple = (42)
21 print("one_val (Tuple):", one_val, type(one_val))
22 print("not_tuple (int):", not_tuple, type(not_tuple))
23

```



```
24 # leeres Tuple
25 empty = ()
26 print("leeres Tuple:", empty, type(empty))
27 print("-")
28
29 # 1) Unveraenderlichkeit (Immutability)
30 print("1) Unveraenderlichkeit (immutable)")
31
32 t = (1, 2, 3)
33 print("t:", t)
34
35 try:
36     # das hier ist NICHT erlaubt
37     t[0] = 99
38 except TypeError as e:
39     print("Fehler bei t[0] = 99:", e)
40
41 print("Man kann ein Tuple nicht in-place aendern - nur NEUE Tuples
42       bauen.")
43 print("-")
44
45 # 2) Zugriff per Index und Slice
46 print("2) Index und Slicing")
47
48 t = (10, 20, 30, 40, 50)
49 print("t:", t)
50 print("t[0]:", t[0]) # 10
51 print("t[-1]:", t[-1]) # 50
52 print("t[1:4]:", t[1:4]) # (20, 30, 40)
53 print("t[::2]:", t[::2]) # (10, 30, 50)
54 print("-")
55
56 # 3) Tuple-Konkatenation und Wiederholung
57 print("3) Konkatenation und Wiederholung")
58
59 a = (1, 2)
60 b = (3, 4)
61 print("a:", a)
62 print("b:", b)
63
64 c = a + b
65 print("a + b:", c) # (1, 2, 3, 4)
66
67 rep = a * 3
68 print("a * 3:", rep) # (1, 2, 1, 2, 1, 2)
69 print("-")
70
71 # 4) Tuple-Unpacking (sehr wichtig!)
72 print("4) Tuple-Unpacking")
73
74 point = (3, 4)
75 x, y = point
76 print("point:", point)
77 print("x:", x)
78 print("y:", y)
79
80 # mit Rest-Stern
81 values = (1, 2, 3, 4, 5)
82 first, *middle, last = values
83 print("values:", values)
```

```
83 print("first:", first)
84 print("middle:", middle) # Liste!
85 print("last:", last)
86 print("-")
87
88 # 5) Rueckgabewerte von Funktionen
89 print("5) Typischer Einsatzzweck: mehrere Rueckgabewerte")
90
91 def min_max(seq):
92     return min(seq), max(seq) # Rueckgabe ist ein Tuple
93
94 nums = [5, 1, 9, 3]
95 res = min_max(nums)
96 print("res:", res, "Typ:", type(res))
97
98 mn, mx = min_max(nums)
99 print("mn:", mn)
100 print("mx:", mx)
101 print("-")
102
103 # 6) Methoden von Tuples: count und index
104 print("6) Methoden: count(x), index(x)")
105
106 t = (1, 2, 2, 3, 2)
107 print("t:", t)
108
109 c = t.count(2)
110 print("t.count(2):", c) # 3
111
112 idx = t.index(2)
113 print("t.index(2):", idx) # 1 (erstes Vorkommen)
114
115 # t.index(99) wuerde ValueError werfen
116 print("-")
117
118 # 7) Tuple vs. Liste - typische Verwendung
119 print("7) Tuple vs. Liste - wann welches?")
120
121 print("- Tuple: unveraenderliche, feste Struktur, oft fuer: ")
122 print(" * Koordinaten (x, y)")
123 print(" * Rueckgabewerte von Funktionen")
124 print(" * Schluessel in Dicts (da hashbar)")
125 print("- Liste: veraenderliche Sammlung, wenn Elemente sich aendern,")
126 print("    wachsen, schrumpfen")
127 print("-")
128
129 # 8) Tuple als Dictionary-Schluessel
130 print("8) Tuple als Dictionary-Schluessel")
131
132 coords = {}
133 coords[(0, 0)] = "Origin"
134 coords[(1, 2)] = "Point A"
135 print("coords:", coords)
136 print("coords[(1, 2)]:", coords[(1, 2)])
137 print("-")
138
139 # 9) Verschachtelte Tuples
140 print("9) Verschachtelte Tuples")
141
142 matrix = (
```

```

142     (1, 2, 3),
143     (4, 5, 6),
144     (7, 8, 9),
145 )
146 print("matrix:")
147 for row in matrix:
148     print(" ", row)
149
150 print("Element in Zeile 2, Spalte 3:", matrix[1][2]) # 6
151 print("-")
152
153 # 10) Umwandlungen zwischen Liste und Tuple
154 print("10) Umwandlung: list() <-> tuple()")
155
156 lst = [1, 2, 3]
157 t = tuple(lst)
158 print("lst:", lst, type(lst))
159 print("tuple(lst):", t, type(t))
160
161 back_to_list = list(t)
162 print("list(t):", back_to_list, type(back_to_list))
163 print("-")
164
165 print("Fertig mit Tuple-Spickzettel.")

```

Listing 4: tuple_spickzettel.py - Komplettes Spickzettel-Skript fuer Tupel

9.2 Kurze uebersicht - Tupel-Eigenschaften & typische Anwendungsfaelle

- **Immutable** -> nach Erstellung unveraenderbar (kein append, kein item assignment, kein sort in-place)
- **Geordnet** -> Zugriff per Index und Slicing moeglich
- **Duplikate erlaubt**
- **Hashbar** -> kann als Schluessel in Dictionaries / Element in Sets verwendet werden
- **Ein-Element-Tupel**: Muss Komma enthalten -> (42,) $\neq$ (42)
- **Typische Einsatzgebiete**:
 - Mehrfach-Rueckgabewerte von Funktionen
 - Feste Datensaeetze (Koordinaten, RGB-Farben, Zeilen in Tabellen, ...)
 - Unpacking (a, b = func())
 - Als Dict-Schluessel oder Set-Element

Wichtigste Methoden (sehr wenige, da immutable):

Methode	Rueckgabewert	Beschreibung
count(x)	int	Anzahl der Vorkommen von x
index(x)	int	Index des ersten Vorkommens - ValueError wenn nicht vorhanden

Typ	Geordnet?	änderbar?	Duplikate erlaubt?	Zugriffsart	Hashbar?	Typische Anwendungsfaelle / Wann waehlen?
list	ja (stabil ab 3.7)	ja (mutable)	ja	Index (int)	nein	veraenderliche Sequenzen, Stapel, Queues, Listen von Objekten, sortierbare Daten
dict	ja (Insert-Reihenfolge ab 3.7 garantiert)	ja	Keys: nein; Values: ja	Schluessel (hashbar)	nein	key → value Zuordnungen, Konfigurationen, schneller Lookup, Alternative zu Named Tuples
set	nein	ja	nein	nur Mitgliedschaft (in)	nein	Einzigkeit erzwingen, Duplikate entfernen, Mengen-Operationen ($\cup$, $\cap$, $-$, Δ), schnelle Existenzpruefung
tuple	ja	nein (immutable)	ja	Index (int)	ja	feste Datensaeetze, Funktions-Rueckgabewerte (mehrere Werte), Dict-/Set-Schluessel, Koordinaten, Records, Unpacking

* Hashbar = kann als Schluessel in einem dict oder als Element in einem set verwendet werden

Methode / Operation	ändert Original?	Rueckgabewert	Time Complexity	Bemerkung / typische Falle
append(x)	ja	None	O(1) amortized	sehr schnell
extend(iterable)	ja	None	O(k)	k = Laenge des Iterables
insert(i, x)	ja	None	O(n)	teuer bei grossem i oder Anfang
remove(x)	ja	None	O(n)	ValueError wenn nicht vorhanden
pop() / pop(-1)	ja	entferntes Element	O(1)	sehr schnell (Ende)
pop(i)	ja	entferntes Element	O(n)	teuer bei i nahe 0
clear()	ja	None	O(1)	seit 3.3
sort(key=None, reverse=False)	ja	None	O(n log n)	in-place, stabil
sorted(iterable, key=..., reverse=...)	nein	neue Liste	O(n log n)	erzeugt Kopie
reverse()	ja	None	O(n)	in-place Umkehrung
copy() / lst[:]	nein	neue Liste	O(n)	shallow copy
count(x)	nein	int	O(n)	
index(x, [start, [end]])	nein	int	O(n)	ValueError wenn nicht gefunden

<code>del lst[i]</code>	ja	-	O(n)	
<code>del lst[a:b]</code>	ja	-	O(n)	Slice loeschen
<code>lst[a:b] = iterable</code>	ja	-	O(n+k)	Slice ersetzen

Methode / Operation	aendert Dict?	Rueckgabewert	Time Complexity	Wichtige Hinweise / Fallen
<code>d[key] = value</code>	ja	-	O(1) avg	ueberschreibt bei Existenz
<code>d[key]</code>	nein	Wert	O(1) avg	KeyError wenn nicht vorhanden
<code>get(key, [default])</code>	nein	Wert oder default	O(1) avg	sicherer Zugriff
<code>setdefault(key, [default])</code>	ja	Wert	O(1) avg	setzt nur wenn nicht vorhanden
<code>update(other) / d = other</code>	ja	None	O(k)	k = Groesse von other
<code>pop(key, [default])</code>	ja	Wert oder default	O(1) avg	KeyError ohne default
<code>popitem()</code>	ja	(key, value)	O(1)	entfernt meist letztes Paar (ab 3.7)
<code>clear()</code>	ja	None	O(1)	
<code>copy()</code>	nein	neues dict	O(n)	shallow copy
<code>keys() / values() / items()</code>	nein	dict view	O(1)	dynamisch, sehr effizient
<code>in / not in</code>	nein	bool	O(1) avg	prueft Schluessel
<code>fromkeys(iterable, [value])</code>	nein	neues dict	O(n)	
<code>del d[key]</code>	ja	-	O(1) avg	KeyError wenn nicht vorhanden

Methode / Operator	aendert Set?	Rueckgabewert	Time Complexity	Bemerkung / Alternative
<code>add(x)</code>	ja	None	O(1) avg	tut nichts, wenn schon vorhanden
<code>update(iterable) / =</code>	ja	None	O(k)	fuegt mehrere Elemente hinzu
<code>discard(x)</code>	ja	None	O(1) avg	kein Fehler, wenn Element fehlt
<code>remove(x)</code>	ja	None	O(1) avg	KeyError, wenn Element fehlt
<code>pop()</code>	ja	Element	O(1) avg	beliebiges Element, Reihenfolge undefiniert
<code>clear()</code>	ja	None	O(1)	leert das Set komplett
<code>copy()</code>	nein	neues Set	O(n)	flache Kopie (shallow)
<code>union / / =</code>	nein / ja	neues Set / None	O(len(s) + len(t))	Mengenvereinigung
<code>intersection / & / &=</code>	nein / ja	neues Set / None	O(min(len(s), len(t)))	Schnittmenge
<code>difference / - / -=</code>	nein / ja	neues Set / None	O(len(s))	Mengendifferenz

<code>symmetric_difference / ^ / ^=</code>	nein / ja	neues Set / None	$O(\text{len}(s) + \text{len}(t))$	symmetrische Differenz
<code>issubset / <= / <</code>	nein	bool	$O(\text{len}(s))$	$< =$ echte Teilmenge
<code>issuperset / >= / ></code>	nein	bool	$O(\text{len}(t))$	analog zu <code>issubset</code>
<code>isdisjoint()</code>	nein	bool	$O(\min(\text{len}(s), \text{len}(t)))$	True, wenn keine gemeinsamen Elemente
<code>frozenset(...)</code>	-	frozenset	-	immutable, hashbar (als Key in <code>dict</code> , Element in <code>set</code>)

10 Wann welchen Collection-Typ waehlen? - Entscheidungshilfe & Beispiele

Diese Section hilft dir, in der Praxis schnell die richtige Wahl zwischen `list`, `dict`, `set` und `tuple` zu treffen.

10.1 Entscheidungsbaum - Schnelluebersicht

Frage	Dann meist die beste Wahl
Brauche ich eine Reihenfolge, die ich behalte und ggf. aendern moechte?	<code>list</code>
Brauche ich eine feste, unveraenderliche Reihenfolge (z. B. Koordinaten, Rueckgabewerte)?	<code>tuple</code>
Brauche ich schnellen Zugriff per Namen/Schluessel (<code>key → value</code>)?	<code>dict</code>
Brauche ich nur Einzigartigkeit und/oder sehr schnelle Existenzpruefung (ist X drin?)?	<code>set</code>
Brauche ich etwas als Schluessel in einem <code>dict</code> oder Element in einem <code>set</code> ?	<code>tuple</code> oder <code>frozenset</code>
Will ich mehrere Werte aus einer Funktion zurueckgeben?	<code>tuple</code> (automatisches Packing/Unpacking)
Will ich Duplikate aus einer Sequenz entfernen, Reihenfolge ist egal?	<code>set</code> (oder <code>dict.fromkeys()</code> ab Py 3.7 fuer Reihenfolge)
Will ich Duplikate entfernen, aber Reihenfolge erhalten?	<code>list(dict.fromkeys(lst))</code> (Py 3.7+)

10.2 Detaillierte Anwendungsfaelle mit Code-Beispielen

10.2.1 1. list - wenn Reihenfolge + aenderbarkeit wichtig sind

Typische Situationen:

- Liste von Usernamen, Aufgaben, Messwerten, Log-Eintraegen
- Stack / Queue (`append` + `pop`)
- Daten, die sortiert, gefiltert, umsortiert werden sollen
- Ich weiss nicht im Voraus, wie viele Elemente kommen

Beispiele:

```

1 # To-Do-Liste
2 todos = ["Einkaufen", "Mail beantworten"]
3 todos.append("Sport")
4 todos.insert(0, "Zahnarzt")           # am Anfang einfüegen
5 todos.sort()                         # alphabetisch sortieren
6 print(todos.pop())                   # letzte Aufgabe erledigt -> '
    Zahnarzt'
7
8 # Messwerte anhaengen
9 temperaturen = []
10 while messung := sensor.read():
11     temperaturen.append(messung)

```

Wann ****nicht**** list?

- Sehr haeufige Existenzpruefung (`x in lst`) -> $O(n)$ -> langsam bei grossen Listen
- Ich brauche etwas als Schluessel in einem dict

10.2.2 2. dict - wenn Zugriff per Schluessel wichtig ist

Typische Situationen:

- Konfigurationsdateien / Einstellungen
- User-Daten (ID -> Profil)
- Zaehlungen / Haeufigkeitszaehler (Counter)
- Gruppierung (z. B. nach Kategorie)
- Schneller Lookup ($O(1)$ average)

Beispiele:

```

1 # User-Profil
2 user = {
3     "id": 4711,
4     "name": "Anna Beispiel",
5     "email": "anna@example.com",
6     "roles": ["admin", "editor"]
7 }
8
9 # Haeufigkeitszaehler
10 from collections import Counter
11 words = ["python", "list", "dict", "python", "set"]
12 count = Counter(words)           # -> Counter({'python': 2, ...})
13
14 # Gruppieren nach Abteilung
15 mitarbeiter = {}
16 for name, abt in daten:
17     mitarbeiter.setdefault(abt, []).append(name)

```

Wann ****nicht**** dict?

- Reihenfolge ist extrem wichtig und ich aendere staendig (dann eher list + dict als Ergaenzung)
- Ich brauche nur Einzigartigkeit ohne Werte -> set ist leichter

10.2.3 3. set - wenn Einzigartigkeit oder Mengenoperationen zaehlen

Typische Situationen:

- Duplikate entfernen
- Schnelle Pruefung „ist X dabei?“ ($O(1)$ average)
- Mengenlehre: Schnittmenge, Vereinigung, Differenz
- Rechte-/Tag-/Kategorie-Mengen vergleichen

Beispiele:

```

1  # Duplikate entfernen, Reihenfolge egal
2  emails = ["a@b.de", "c@d.de", "a@b.de", "e@f.de"]
3  unique_emails = list(set(emails))           # oder set(emails) wenn
        Reihenfolge egal
4
5  # Gemeinsame Rechte
6  admin_rechte = {"read", "write", "delete", "admin"}
7  user_rechte = {"read", "write"}
8  print(admin_rechte & user_rechte)           # {'read', 'write'}
9  print(admin_rechte - user_rechte)           # {'delete', 'admin'}
10
11 # Schnelle Pruefung
12 if "admin" in user_rechte:
13     print("Hat Admin-Rechte")

```

Wann ****nicht**** set?

- Reihenfolge ist wichtig
- Ich brauche Werte zu den Elementen -> dict

10.2.4 4. tuple - wenn Daten fest / unveraenderlich sein sollen

Typische Situationen:

- Koordinaten (x, y, z)
- RGB-Farben (r, g, b)
- Rueckgabewerte mehrerer Werte aus Funktionen
- Als Schluessel in dict oder Element in set
- Feste Datensaeetze / Records (vor namedtuple/dataclass)

Beispiele:

```

1  # Koordinaten
2  punkt = (3.14, 2.718)
3  x, y = punkt                               # unpacking
4
5  # Funktion mit mehreren Rueckgabewerten
6  def minmax(lst):
7      return min(lst), max(lst)
8
9  minimum, maximum = minmax(temperaturen)
10
11 # Als dict-Schluessel
12 entfernungen = {}
13 entfernungen[( "Berlin", "Muenchen" )] = 585

```



```
14 | entfernungen[( "Hamburg", "Koeln" )] = 430
```

Wann ****nicht**** tuple?

- Die Daten sollen sich aendern koennen
- Ich muss Elemente anhaengen/entfernen/sortieren

10.3 Zusammenfassung - typische Kombinationen in der Praxis

- **Duplikate entfernen + Reihenfolge behalten** (Py 3.7+): `list(dict.fromkeys(meine_liste))`
- **Zugriff per Index + per Name**: `list` + `dict` (oder `dataclasses` / `namedtuple`)
- **Gruppieren + Zaehlen**: `dict` mit `list`- oder `set`-Werten oder `defaultdict`
- **Mitgliedschaft + Operationen**: `set` oder `frozenset` (wenn hashbar benoetigt)
- **Feste Struktur + Hashbar**: `tuple` oder `frozenset`

11 Verschachtelte Collections (Nested Structures) - Wichtig & tueckisch

In der Praxis kommen fast nie „flache“ Listen/Dicts/Sets/Tupel vor. Sehr haeufig hat man:

- Listen in Listen (2D-Arrays, Matrizen, Tabellen)
- Dictionaries in Listen (Liste von User-Profilen)
- Listen/Dicts in Dictionaries (Gruppierungen, JSON-aehnliche Strukturen)
- Tupel in Listen oder als Dict-Werte/Schluessel

Das fuehrt zu einem wichtigen Prinzip: **shallow vs. deep copy** und **Referenzprobleme**.

11.1 1. Das klassische Problem: Shallow Copy reicht nicht

```
1  # Achtung: flache Kopie!
2  users = [
3      {"name": "Anna", "scores": [85, 92, 78]},
4      {"name": "Ben", "scores": [64, 88, 91]}
5  ]
6
7  # Flache Kopie (default bei copy() oder [:])
8  import copy
9  users_copy = users.copy()          # oder users[:]
10
11 # Aenderung an einem inneren Objekt
12 users_copy[0]["scores"].append(95)
13 users_copy[0]["name"] = "Anna Neu"
14
15 print(users[0]["scores"])           # [85, 92, 78, 95] <-- Original
    wurde mitgeaendert!
16 print(users[0]["name"])             # "Anna" <-- aeussere Ebene
    ist ok
```

Listing 5: Klassischer Fehler bei verschachtelten Strukturen

Fazit: `.copy()` / `[:]` / `dict.copy()` kopieren nur die **aeussere** Collection. Innere mutable Objekte (`list`, `dict`, `set`) bleiben dieselben Referenzen!

11.2 2. Loesung: Deep Copy

```

1 import copy
2
3 users_deep = copy.deepcopy(users)
4
5 # Jetzt sicher aendern
6 users_deep[0]["scores"].append(100)
7 users_deep[1]["name"] = "Ben Neu"
8
9 print(users[0]["scores"])          # [85, 92, 78]      <-- Original
                                   unveraendert!

```

Wann deepcopy vermeiden?

- Sehr grosse/komplexe Strukturen -> kann langsam sein
- Zirkulaere Referenzen -> kann Endlosschleife erzeugen (selten)
- Man braucht bewusst geteilte Referenzen (z. B. gemeinsame Konfig-Objekte)

11.3 3. Typische verschachtelte Strukturen - Beispiele

11.3.1 Liste von Dictionaries (sehr haeufig: JSON-aehnlich)

```

1 people = [
2     {"id": 1, "name": "Alice", "hobbies": ["lesen", "wandern"]},
3     {"id": 2, "name": "Bob", "hobbies": ["gamen", "sport"]},
4 ]
5
6 # Neuen Eintrag hinzufuegen
7 people.append({"id": 3, "name": "Charlie", "hobbies": []})
8
9 # Hobby hinzufuegen
10 people[1]["hobbies"].append("kochen")
11
12 print(people[1])    # {'id': 2, 'name': 'Bob', 'hobbies': ['gamen', 'sport', 'kochen']}

```

11.3.2 Dictionary mit Listen als Werten (Gruppierung)

```

1 from collections import defaultdict
2
3 gruppen = defaultdict(list)
4
5 daten = [("rot", "Apfel"), ("gruen", "Kiwi"), ("rot", "Tomate"), ("gelb", "Banane")]
6
7 for farbe, frucht in daten:
8     gruppen[farbe].append(frucht)
9
10 print(gruppen)
11 # defaultdict(<class 'list'>, {'rot': ['Apfel', 'Tomate'], 'gruen': ['Kiwi'], 'gelb': ['Banane']})

```

11.3.3 Matrix (Liste von Listen)

```

1 # 3 x 4 Matrix
2 matrix = [

```

```

3      [1,  2,  3,  4],
4      [5,  6,  7,  8],
5      [9, 10, 11, 12]
6  ]
7
8  # Zeile 2, Spalte 3 auslesen (0-basiert!)
9  print(matrix[1][2])          # 7
10
11 # Wert aendern
12 matrix[0][3] = 99
13 print(matrix[0])              # [1, 2, 3, 99]
14
15 # Vorsicht bei Initialisierung!
16 # FALSCH (alle Zeilen sind dieselbe Referenz!):
17 # bad = [[0]*4]*3
18
19 # RICHTIG:
20 good = [[0]*4 for _ in range(3)]

```

11.3.4 Tupel als Schluessel in Dicts (sehr nuetzlich)

```

1 punkte = {}
2 punkte[(10, 20)] = "A"
3 punkte[(15, 35)] = "B"
4
5 print(punkte[(10, 20)])      # "A"
6
7 # Tupel mit Listen als Schluessel -> TypeError!
8 # punkte[[1,2]] = "fehler"   # TypeError: unhashable type: 'list'

```

11.4 4. Schnelle Merkgeregeln fuer verschachtelte Collections

Situation	Beste Praxis / Empfehlung
Ich kopiere eine nested list/dict	<code>copy.deepcopy()</code> oder manuell neu aufbauen
Ich will eine Matrix erzeugen	<code>[[0]*cols for _ in range(rows)]</code> - nie <code>[[0]*cols]*rows</code>
Ich gruppriere Daten	<code>defaultdict(list)</code> oder <code>dict.setdefault(key, []).append(value)</code>
Ich brauche feste innere Strukturen	<code>tuple</code> oder <code>namedtuple</code> / <code>dataclass</code> / <code>TypedDict</code>
Ich habe sehr tiefe Verschachtelung	<code>dataclasses</code> , <code>pydantic</code> , <code>attrs</code> oder <code>JSON</code>

11.5 5. Zusammenfassung - typische Fehlerquellen

- `.copy()` / `[:]` bei nested mutable Objekten -> unbeabsichtigte aenderungen
- `[[0]*n]*m` -> alle Zeilen sind dieselbe Liste
- Listen / Dicts / Sets als Schluessel -> `TypeError: unhashable type`
- Vergessen, dass `defaultdict` / `setdefault` mutable Defaults zurueckgeben
- `deepcopy` bei sehr grossen Daten -> Performance-Problem